



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №8

із дисципліни «Технології розроблення програмного забезпечення»

Тема: «Патерни проектування»

Тема проекту: «Музичний програвач»

GitHub: <https://github.com/martyshenkoo/MusicPlayerWeb>

Виконала:

студентка групи ІА-31:

Мартишенко І.С.

Перевірив:

Мягкий М.Ю.

Мета: Вивчити структуру шаблонів «Composite», «Flyweight» (Пристосуванець), «Interpreter», «Visitor» та навчитися застосовувати їх в реалізації програмної системи.

Музичний програвач (iterator, command, memento, facade, visitor, client-server)

Музичний програвач становить собою програму для програвання музичних файлів або відтворення потокової музики з можливістю створення, запам'ятовування і редагування списків програвання, перемішування/повторення (shuffle/repeat), розпізнавання різних аудіо-форматів, еквалайзер.

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Теоретичні відомості

Призначення: Шаблон відвідувач дозволяє вказувати операції над елементами без зміни структури конкретних елементів. Таким чином вкрай зручно додавати нові операції, проте дуже важко додавати нові елементи в ієрархію (необхідно додавати відповідні методи для обробки їх відвідувань в кожного відвідувача).

Даний шаблон дозволяє групувати однотипні операції, що застосовуються над різнотипними об'єктами.

Проблема: Ви розробляєте онлайн-корзину інтернет магазину. Товари які представлені в магазині є різних типів, наприклад, електроніка, міцні напої, домашня хімія.

Логіка роботи з товарами в корзині є різна, наприклад, розрахунок вартості, формування замовлення.

Ми можемо всі ці методи зробити в товарах, але тоді ми ускладнюємо товари і змішуємо логіку (розрахунок вартості) з даними (товаром та його кількістю).

Якщо ми в подальшому необхідно буде додати ще логіку розрахунку вартості з врахуванням знижки, то потрібно буде додати ще цю логіку до товарів. А якщо буде ще сезонна знижка, то ми знову будемо додавати нову логіку до класів товарів.

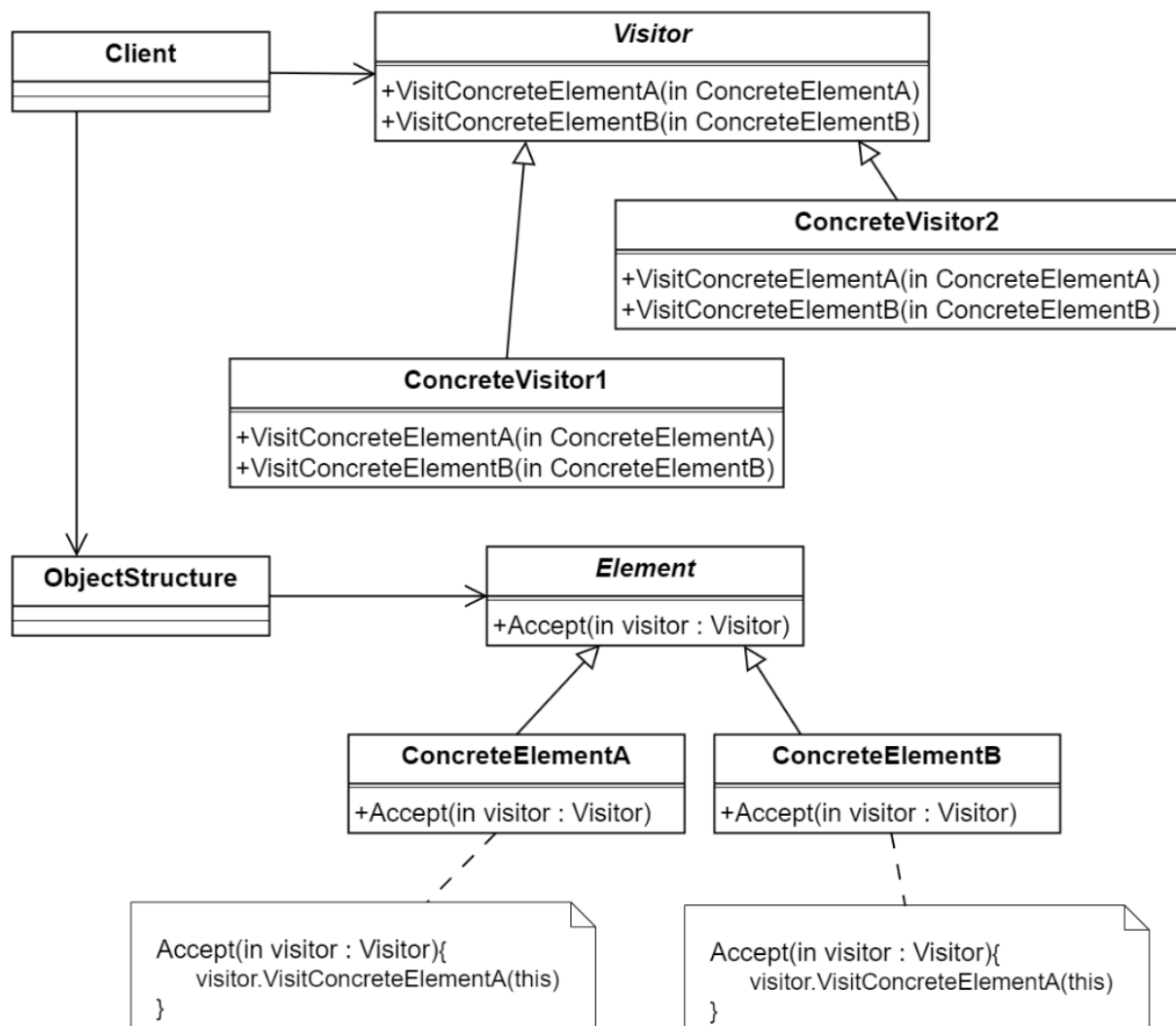


Рис-1. Структура патерну «Visitor»

Рішення: Основна ідея патерна «Відвідувач» це рознести логіку і дані в різні класи та ієрархії. Якщо так зробити для нашої онлайн-корзини, то в класі

відвідувача ми маємо функції для обрахунку логіки для кожного типу, а об'єкти в корзині знають свій тип, отримують екземпляр відвідувача і в нього викликають метод відповідно до свого типу. В результаті ми робимо різні відвідувачі для розрахунку вартості: один для звичайних розрахунків, інший для розрахунку вартості зі знижкою, ще один для розрахунку сезонних знижок.⁹⁷ За рахунок того, що логіка відокремлена від наших товарів в корзині ми можемо реалізовувати за необхідності нові класи відвідувачі, а класи товарів та і корзини, в цілому, змінюватися не будуть.

Якщо розвивати далі, то можна зробити і клас відвідувача, який буде формувати замовлення з товарів в корзині в залежності від продавців і можливих варіантів доставки. Це дозволить формувати, наприклад, не одне замовлення, а два одна з самовивозом з магазину, а інше з доставкою Новою Поштою.

Приклад з життя: Прикладом може служити написання компілятора.

Припустимо, існують різні об'єкти в синтаксисі мови програмування: виклики методів і умовні вирази. Компілятор перед генерацією коду повинен обійти всі вирази (і виклики методів, і умовні вирази) і перевірити безпеку типів, після чого згенерувати відповідний код. Відповідно буде два відвідувачі – для перевірки безпеки типів і для генерації коду. У кожного з них буде по 2 методи – для викликів методів і для умовних операцій. Таким чином при необхідності додавання нових кроків компіляції досить буде визначити нового «відвідувача» і викликати його у відповідний час.

Хід роботи

У межах проєкту «Музичний програвач» було реалізовано основну логіку роботи з користувачами та аудіотреками.

Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

```

1  using System.Linq;
2
3  namespace MusicPlayerWeb.Models;
4
5  public class Playlist : IPlaylistVisitable
6  {
7      public Guid Id { get; set; } = Guid.NewGuid();
8      public string Title { get; set; } = default!;
9      public string OwnerUsername { get; set; } = default!;
10     public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
11
12     public ICollection<PlaylistTrack> PlaylistTracks { get; set; } = new List<PlaylistTrack>();
13
14     public ITrackIterator CreateIterator(IEnumerable<Track> tracks)
15     {
16         return new PlaylistIterator(tracks);
17     }
18
19     public void Accept(IPlaylistVisitor visitor)
20     {
21         visitor.VisitPlaylist(this);
22
23         foreach (var track in PlaylistTracks.Select(pt => pt.Track).Where(t => t != null))
24         {
25             track.Accept(visitor);
26         }
27     }
28 }

```

Рис-1. Models – Playlist.cs

Models > C# Track.cs

```

1  namespace MusicPlayerWeb.Models;
2
3  public class Track : IPlaylistVisitable
4  {
5      public Guid Id { get; set; } = Guid.NewGuid();
6      public string Title { get; set; } = default!;
7      public string FileName { get; set; } = default!;
8      public string RelativeUrl { get; set; } = default!;
9      public DateTime AddedAt { get; set; } = DateTime.UtcNow;
10     public string? OwnerUsername { get; set; }
11
12     public void Accept(IPlaylistVisitor visitor)
13     {
14         visitor.VisitTrack(this);
15     }
16 }

```

Рис-2. Models – Track.cs

Services > Visitors > C# PlaylistStatisticsVisitor.cs

```
1  using System.Collections.Generic;
2  using MusicPlayerWeb.Models;
3
4  namespace MusicPlayerWeb.Services.Visitors;
5
6  public class PlaylistStatisticsVisitor : IPlaylistVisitor
7  {
8      private readonly List<string> _titles = new();
9
10     public string? PlaylistTitle { get; private set; }
11     public int TrackCount { get; private set; }
12
13     public void VisitPlaylist(Playlist playlist)
14     {
15         PlaylistTitle = playlist.Title;
16     }
17
18     public void VisitTrack(Track track)
19     {
20         TrackCount++;
21         _titles.Add(track.Title);
22     }
23
24     public PlaylistStatistics ToStatistics()
25     {
26         return new PlaylistStatistics
27         {
28             PlaylistTitle = PlaylistTitle,
29             TrackCount = TrackCount,
30             TrackTitles = _titles.AsReadOnly()
31         };
32     }
33 }
34
```

Рис-3. Services – Visitors – PlaylistStatisticsVisitor.cs

```

1  using System;
2  using MusicPlayerWeb.Models;
3
4  namespace MusicPlayerWeb.Services.Visitors;
5
6  public class PlaylistVisitorService
7  {
8      private readonly IPlaylistService _playlists;
9
10     public PlaylistVisitorService(IPlaylistService playlists)
11     {
12         _playlists = playlists;
13     }
14
15     public PlaylistStatistics GetStatistics(string username, Guid playlistId)
16     {
17         var playlist = _playlists.GetById(username, playlistId);
18         if (playlist == null)
19             return PlaylistStatistics.Empty;
20
21         var visitor = new PlaylistStatisticsVisitor();
22         playlist.Accept(visitor);
23         return visitor.ToStatistics();
24     }
25 }

```

Рис-4. Services – Visitors – PlaylistService.cs

Реалізувати один з розглянутих шаблонів за обраною темою.

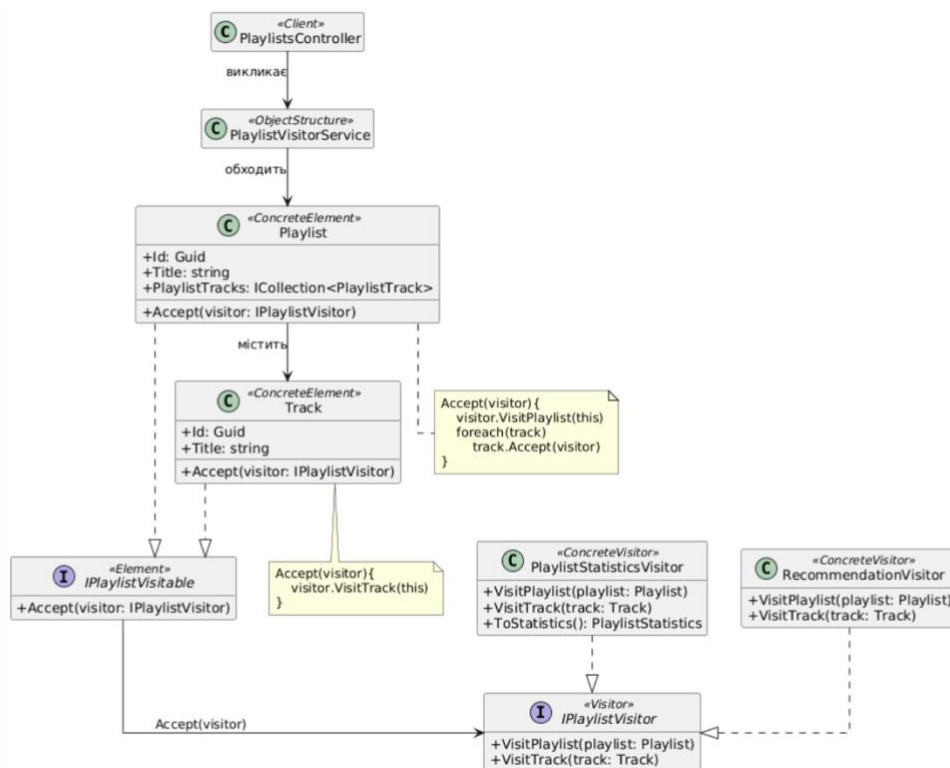


Рис-5. Структура патерну Visitor

Client (PlaylistsController).

Контролер ініціює користувацькі сценарії. Наприклад, коли користувач відкриває сторінку статистики плейліста, контролер звертається до PlaylistVisitorService, не взаємодіючи напяму з доменними моделями.

ObjectStructure (PlaylistVisitorService).

Сервіс отримує потрібний плейліст через IPlaylistService, проходить по його структурі та оркеструє виклики методу Асепт у всіх елементів. Він не містить бізнес-логіки, а лише забезпечує коректний обхід доменної структури.

Element / IPlaylistVisitable.

Це контракт, який гарантує, що кожна доменна сутність (Playlist, Track) здатна «прийняти» візитора. Тут реалізується механізм подвійної диспетчеризації: елемент знає, який саме метод Visit* слід викликати для конкретного типу об'єкта.

ConcreteElements (Playlist, Track).

- Playlist у методі Асепт спочатку передає себе у visitor.VisitPlaylist(this), після чого послідовно викликає Асепт у всіх власних треків.
- Track реалізує простий виклик visitor.VisitTrack(this).

Завдяки цьому вся структура плейліста відкривається зовнішньому візитору без необхідності модифікувати контролер або порушувати інкапсуляцію.

Visitor / IPlaylistVisitor.

Інтерфейс визначає операції, які можуть застосовуватись до елементів плейліста. Він містить методи VisitPlaylist і VisitTrack, примушуючи кожного нового візитора визначити поведінку для обох типів елементів.

ConcreteVisitors (PlaylistStatisticsVisitor та інші).

PlaylistStatisticsVisitor отримує назву плейліста, підраховує кількість треків і формує DTO PlaylistStatistics. На діаграмі також наведений приклад потенційного RecommendationVisitor, який міг би виконувати іншу операцію (наприклад, формувати

рекомендації чи перевіряти ліцензійні обмеження). Нові візитори можуть підключатися без будь-яких змін у моделях Playlist чи Track.

Послідовність викликів.

Показаний ланцюжок взаємодій: контролер → сервіс → плейліст → треки. У кожному кроці використовується пара методів Accept/Visit. Це дозволяє додавати нові операції через реалізацію нових візиторів без зміни існуючих доменних моделей чи клієнтського коду.

Такий підхід відповідає класичній реалізації патерну *Visitor* та демонструє чітке розділення структури (плейліст і треки) та поведінки (візитори).

```
Models > C# IPlaylistVisitor.cs
1  namespace MusicPlayerWeb.Models;
2
3  public interface IPlaylistVisitor
4  {
5      void VisitPlaylist(Playlist playlist);
6      void VisitTrack(Track track);
7  }
8
9  public interface IPlaylistVisitable
10 {
11     void Accept(IPlaylistVisitor visitor);
12 }
13
```

Рис-6. Models – IPlaylistVisitor.cs

Models > C# Playlist.cs

```
1  using System.Linq;
2
3  namespace MusicPlayerWeb.Models;
4
5  ///
6  public class Playlist : IPlaylistVisitable
7  {
8      public Guid Id { get; set; } = Guid.NewGuid();
9      public string Title { get; set; } = default!;
10     public string OwnerUsername { get; set; } = default!;
11     public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
12
13     public ICollection<PlaylistTrack> PlaylistTracks { get; set; } = new List<PlaylistTrack>();
14
15     public ITrackIterator CreateIterator(IEnumerable<Track> tracks)
16     {
17         return new PlaylistIterator(tracks);
18     }
19
20     public void Accept(IPlaylistVisitor visitor)
21     {
22         visitor.VisitPlaylist(this);
23
24         foreach (var track in PlaylistTracks.Select(pt => pt.Track).Where(t => t != null))
25         {
26             track.Accept(visitor);
27         }
28     }
29 }
```

Рис-7. Models – Playlist.cs

Models > C# Track.cs

```
1  namespace MusicPlayerWeb.Models;
2
3  ///
4  public class Track : IPlaylistVisitable
5  {
6      public Guid Id { get; set; } = Guid.NewGuid();
7      public string Title { get; set; } = default!;
8      public string FileName { get; set; } = default!;
9      public string RelativeUrl { get; set; } = default!;
10     public DateTime AddedAt { get; set; } = DateTime.UtcNow;
11     public string? OwnerUsername { get; set; }
12
13     public void Accept(IPlaylistVisitor visitor)
14     {
15         visitor.VisitTrack(this);
16     }
17 }
```

Рис-8. Models – Track.cs

Services > Visitors > C# PlaylistStatisticsVisitor.cs

```
1  using System.Collections.Generic;
2  using MusicPlayerWeb.Models;
3
4  namespace MusicPlayerWeb.Services.Visitors;
5
6  public class PlaylistStatisticsVisitor : IPlaylistVisitor
7  {
8      private readonly List<string> _titles = new();
9
10     public string? PlaylistTitle { get; private set; }
11     public int TrackCount { get; private set; }
12
13     public void VisitPlaylist(Playlist playlist)
14     {
15         PlaylistTitle = playlist.Title;
16     }
17
18     public void VisitTrack(Track track)
19     {
20         TrackCount++;
21         _titles.Add(track.Title);
22     }
23
24     public PlaylistStatistics ToStatistics()
25     {
26         return new PlaylistStatistics
27         {
28             PlaylistTitle = PlaylistTitle,
29             TrackCount = TrackCount,
30             TrackTitles = _titles.AsReadOnly()
31         };
32     }
33 }
```

Рис-9. Services – Visitors – PlaylistStatisticsVisitor.cs

```

Services > Visitors > C# PlaylistVisitorService.cs
1  using System;
2  using MusicPlayerWeb.Models;
3
4  namespace MusicPlayerWeb.Services.Visitors;
5
6  public class PlaylistVisitorService
7  {
8      private readonly IPlaylistService _playlists;
9
10     public PlaylistVisitorService(IPlaylistService playlists)
11     {
12         _playlists = playlists;
13     }
14
15     public PlaylistStatistics GetStatistics(string username, Guid playlistId)
16     {
17         var playlist = _playlists.GetById(username, playlistId);
18         if (playlist == null)
19             return PlaylistStatistics.Empty;
20
21         var visitor = new PlaylistStatisticsVisitor();
22         playlist.Accept(visitor);
23         return visitor.ToStatistics();
24     }
25 }
26

```

Рис-10. Services – Visitors – PlaylistVisitorService.cs

Реалізувати не менше 3-х класів відповідно до обраної теми.

```

Models > C# PlaylistTrack.cs
1  namespace MusicPlayerWeb.Models;
2
3  public class PlaylistTrack
4  {
5      public Guid Id { get; set; } = Guid.NewGuid();
6
7      public Guid PlaylistId { get; set; }
8      public Playlist Playlist { get; set; } = default!;
9
10     public Guid TrackId { get; set; }
11     public Track Track { get; set; } = default!;
12
13     public int Order { get; set; }
14 }
15

```

Рис-11. Models – PlaylistTrack.cs

```

service.cs  C# IPlaylistVisitor.cs  C# Playlist.cs  C# Track.cs  C# PlaylistStatisticsVisitor.cs  C# PlaylistVisitors
Models > C# PlaylistStatistics.cs
1  using System;
2  using System.Collections.Generic;
3
4  namespace MusicPlayerWeb.Models;
5
6  public class PlaylistStatistics
7  {
8      public string? PlaylistTitle { get; init; }
9      public int TrackCount { get; init; }
10     public IReadOnlyCollection<string> TrackTitles { get; init; } = Array.Empty<string>();
11
12     public static PlaylistStatistics Empty => new PlaylistStatistics
13     {
14         PlaylistTitle = null,
15         TrackCount = 0,
16         TrackTitles = Array.Empty<string>()
17     };
18 }
19

```

Рис-12. Models – PlaylistStatistics.cs

```

Services > Visitors > C# PlaylistVisitorService.cs
1  using System;
2  using MusicPlayerWeb.Models;
3
4  namespace MusicPlayerWeb.Services.Visitors;
5
6  public class PlaylistVisitorService
7  {
8      private readonly IPlaylistService _playlists;
9
10     public PlaylistVisitorService(IPlaylistService playlists)
11     {
12         _playlists = playlists;
13     }
14
15     public PlaylistStatistics GetStatistics(string username, Guid playlistId)
16     {
17         var playlist = _playlists.GetById(username, playlistId);
18         if (playlist == null)
19             return PlaylistStatistics.Empty;
20
21         var visitor = new PlaylistStatisticsVisitor();
22         playlist.Accept(visitor);
23         return visitor.ToStatistics();
24     }
25 }
26

```

Рис-13. Services – Visitors – PlaylistVisitorService.cs

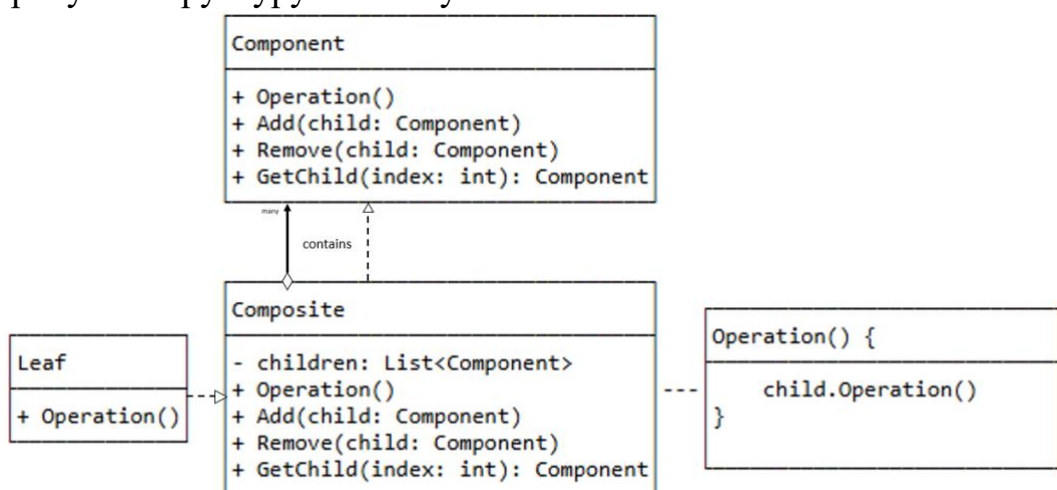
Висновок:

У ході виконання лабораторної роботи я зосередилася на застосуванні патерну *Visitor* у контексті музичного програвача. Було реалізовано метод *Асепт* у доменних моделях

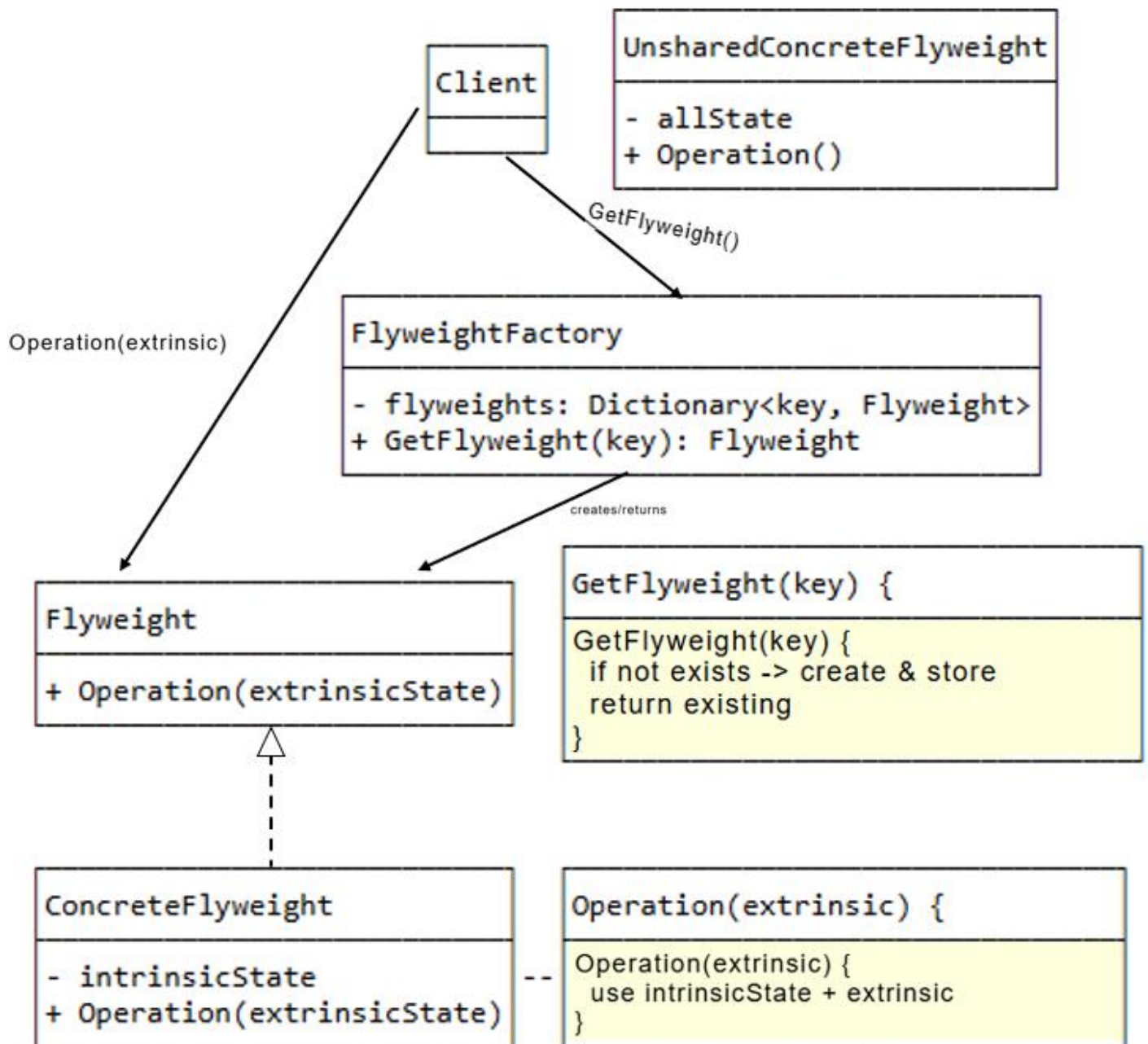
Playlist і Track, створено візитор PlaylistStatisticsVisitor, а також сервіс, що відповідає за його виклик. Така практична реалізація дала змогу чітко зрозуміти призначення патерну: він забезпечує можливість додавання нових операцій над плейлістами та треками (наприклад, формування статистики, аналітика чи рекомендації) без внесення змін у самі доменні класи. Це підвищує гнучкість архітектури, спрощує розширення функціональності та покращує підтримуваність системи.

Відповіді на контрольні запитання:

1. Яке призначення шаблону «Композит»? Шаблон використовується для складання об'єктів у деревоподібну структуру для подання ієрархій типу «частина - ціле». Дозволяє уніфіковано обробляти поодинокі об'єкти та композиції.
2. Нарисуйте структуру шаблону «Композит».



3. Які класи входять в шаблон «Композит», та яка між ними взаємодія?
 - **Component** - абстрактний клас/інтерфейс з операціями.
 - **Leaf** - кінцевий елемент, реалізує операції.
 - **Composite** - контейнер, містить children, рекурсивно делегує операції. Взаємодія: клієнт працює з **Component**, не розрізняючи **Leaf** і **Composite**.
4. Яке призначення шаблону «Легковаговик»? Зменшення кількості об'єктів у пам'яті шляхом поділу спільного (внутрішнього) стану між кількома екземплярами.
5. Нарисуйте структуру шаблону «Легковаговик».



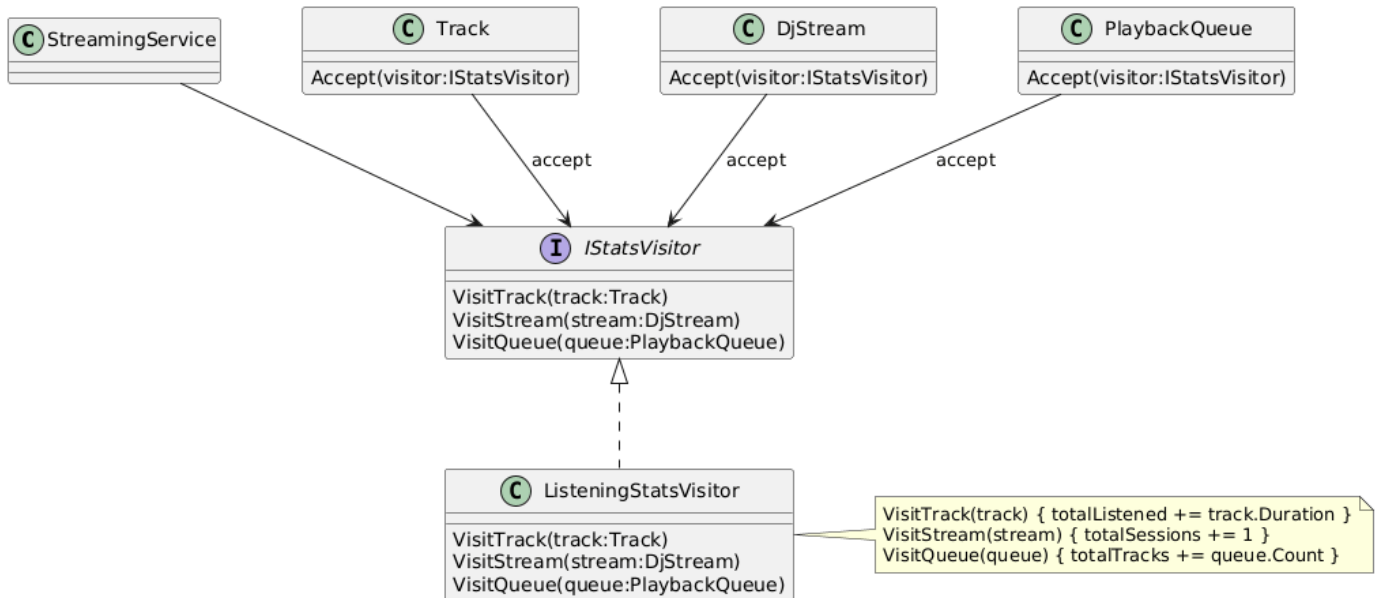
6. Які класи входять в шаблон «Легковаговик», та яка між ними взаємодія?

- Flyweight - інтерфейс з операцією, що приймає зовнішній стан.
- ConcreteFlyweight - зберігає внутрішній стан, спільний.
- UnsharedConcreteFlyweight - не поділюваний варіант.
- FlyweightFactory - кешує та повертає Flyweight за ключем. Взаємодія: клієнт отримує Flyweight з фабрики, передає зовнішній стан у метод.

7. Яке призначення шаблону «Інтерпретатор»? Подання граматики мови та інтерпретатора для обчислення виразів у термінах абстрактного синтаксичного дерева (AST).

8. Яке призначення шаблону «Відвідувач»? Дозволяє додавати нові операції над елементами ієрархії без зміни їх класів, відокремлюючи логіку від структури.

9. Нарисуйте структуру шаблону «Відвідувач».



10. Які класи входять в шаблон «Відвідувач», та яка між ними взаємодія?

- Visitor - інтерфейс з методами Visit... для кожного типу елемента.
- ConcreteVisitor - реалізує операції.
- Element - інтерфейс з Accept(Visitor).
- ConcreteElement - викликає відповідний Visit... у відвідувача.
- ObjectStructure - колекція елементів. Взаємодія: елемент викликає visitor.Visit(this), відвідувач виконує логіку за типом.